Status	Finished
Started	Thursday, 8 October 2026, 11:34 AM
Completed	Thursday, 8 October 2026, 7:30 PM
Duration	7 hours 56 mins
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

The program must accept **N** series of keystrokes as string values as the input. The character **^** represents undo action to clear the last entered keystroke. The program must print the string typed after applying the undo operations as the output. If there are no characters in the string then print **-1** as the output.

Boundary Condition(s):

1 <= N <= 100
1 <= Length of each string <= 100

Input Format:

The first line contains the integer N.
The next N lines contain a string on each line.

Output Format:

The first N lines contain the string after applying the undo operations.

Example Input/Output 1:

Input:

3
Hey ^ goooo^^glee^
lucke^y ^charr^ms
ora^^nge^^^^

Output:

Hey google
luckycharms
-1

Answer: (penalty regime: 0 %)

```
n = int(input())

for i in range(n):
    s = input()
    ans = ""

    for ch in s:
        if ch == '^':
            if len(ans) > 0:
                ans = ans[:-1]
        else:
            ans = ans + ch

    if ans == "":
        print(-1)
    else:
        print(ans)
```

	Input	Expected	Got	
✓	3 Hey ^ goooo^^glee^ lucke^y ^charr^ms ora^^nge^^^^	Hey google luckycharms -1	Hey google luckycharms -1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Consider the below words as key words and check the given input is key word or not.

keywords: {break, case, continue, default, defer, else, for, func, goto, if, map, range, return, struct, type, var}

Input format:

Take string as an input from stdin.

Output format:

Print the word is key word or not.

Example Input:

break

Output:

break is a keyword

Example Input:

IF

Output:

IF is not a keyword

For example:

Input	Result
break	break is a keyword
IF	IF is not a keyword

Answer: (penalty regime: 0 %)

```
keywords = ["break", "case", "continue", "default", "defer",
            "else", "for", "func", "goto", "if", "map",
            "range", "return", "struct", "type", "var"]

s = input()

if s in keywords:
    print(s, "is a keyword")
else:
    print(s, "is not a keyword")
```

	Input	Expected	Got	
✓	break	break is a keyword	break is a keyword	✓
✓	IF	IF is not a keyword	IF is not a keyword	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

A pangram is a sentence where every letter of the English alphabet appears at least once.

Given a string sentence containing only lowercase English letters, return true if sentence is a pangram, or false otherwise.

Example 1:

Input:

thequickbrownfoxjumpsoverthelazydog

Output:

true

Explanation: sentence contains at least one of every letter of the English alphabet.

Example 2:

Input:

arvijayakumar

Output: false

Constraints:

1 <= sentence.length <= 1000

sentence consists of lowercase English letters.

For example:

Test	Result
`print(checkPangram('thequickbrownfoxjumpsoverthelazydog'))`	`true`
`print(checkPangram('arvijayakumar'))`	`false`

Answer: (penalty regime: 0 %)

Reset answer

```
def checkPangram(sentence):
    # one slot per letter: seen[0] is 'a', seen[25] is 'z'
    seen = [False] * 26

    # index-based loop over the sentence
    for i in range(len(sentence)):
        # map the letter to 0..25, e.g. 'c' -> 2
        idx = ord(sentence[i]) - ord('a')
        seen[idx] = True

    # every letter must have been seen at least once
    for i in range(26):
        if not seen[i]:
            return "false"

    return "true"
```

	Test	Expected	Got	
✓	`print(checkPangram('thequickbrownfoxjumpsoverthelazydog'))`	`true`	`true`	✓

	Test	Expected	Got	
✓	`print(checkPangram('arvijayakumar'))`	`false`	`false`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Given a **non-empty** string s and an abbreviation abbr, return whether the string matches with the given abbreviation.

A string such as "word" contains only the following valid abbreviations:

["word", "1ord", "w1rd", "wo1d", "wor1", "2rd", "w2d", "wo2", "1o1d", "1or1", "w1r1", "1o2", "2r1", "3d", "w3", "4"]

Notice that only the above abbreviations are valid abbreviations of the string "word". Any other string is not a valid abbreviation of "word".

Note:

Assume s contains only lowercase letters and abbr contains only lowercase letters and digits.

Example 1:

Input

internationalization
i12iz4n

Output

true

Explanation

Given **s** = "internationalization", **abbr** = "i12iz4n":

Return true.

Example 2:

Input

apple
a2e

Output

false

Explanation

Given **s** = "apple", **abbr** = "a2e":

Return false.

Answer: (penalty regime: 0 %)

```
s = input().strip()
abbr = input().strip()

i = 0
j = 0
n = len(s)
m = len(abbr)
ok = True

while i < n and j < m:
    c = abbr[j]
    if c.isdigit():
        if c == '0':
            ok = False
            break
        num = 0
        while j < m and abbr[j].isdigit():
            num = num * 10 + int(abbr[j])
            j += 1
        i += num
    else:
        if s[i] != c:
            ok = False
            break
        i += 1
        j += 1

if ok and i == n and j == m:
    print("true")
else:
    print("false")

```

	Input	Expected	Got	
✓	internationalization i12iz4n	true	true	✓
✓	apple a2e	false	false	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Find if a String2 is substring of String1. If it is, return the index of the first occurrence. else return -1.

Sample Input 1

thistest123string

123

Sample Output 1

8

Answer: (penalty regime: 0 %)

```
s1 = input().strip()
s2 = input().strip()

n = len(s1)
m = len(s2)
ans = -1

# try every possible starting index in s1
for i in range(n - m + 1):
    # compare s2 against s1 starting at i
    match = True
    for j in range(m):
        if s1[i + j] != s2[j]:
            match = False
            break
    # first full match is the answer
    if match:
        ans = i
        break

print(ans)
```

	Input	Expected	Got	
✓	thistest123string 123	8	8	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

Given a string s containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

An input string is valid if:

Open brackets must be closed by the same type of brackets.

Open brackets must be closed in the correct order.

Constraints:

1 <= s.length <= 10^4

s consists of parentheses only '()[]{}'.

For example:

Test	Result
`print(ValidParenthesis("()"))`	`true`
`print(ValidParenthesis("()[]{}"))`	`true`
`print(ValidParenthesis("(]"))`	`false`

Answer: (penalty regime: 0 %)

Reset answer

```
def ValidParenthesis(s):
    # stack holds the opening brackets still waiting for a partner
    stack = []

    # index-based loop over the string
    for i in range(len(s)):
        c = s[i]

        if c == '(' or c == '{' or c == '[':
            # opening bracket: push it
            stack.append(c)
        else:
            # closing bracket: stack must not be empty
            if len(stack) == 0:
                return "false"

            # the most recent opener must be the matching type
            top = stack.pop()
            if c == ')' and top != '(':
                return "false"
            if c == '}' and top != '{':
                return "false"
            if c == ']' and top != '[':
                return "false"

    # valid only if every opener got closed
    if len(stack) == 0:
        return "true"
    return "false"
```

	Test	Expected	Got	
✓	`print(ValidParenthesis("()"))`	true	true	✓
✓	`print(ValidParenthesis("()[]{}"))`	true	true	✓
✓	`print(ValidParenthesis("(]"))`	false	false	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Given a string S which is of the format USERNAME@DOMAIN.EXTENSION, the program must print the EXTENSION, DOMAIN, USERNAME in the reverse order.

Input Format:

The first line contains S.

Output Format:

The first line contains EXTENSION.
The second line contains DOMAIN.
The third line contains USERNAME.

Boundary Condition:

1 <= Length of S <= 100

Example Input/Output 1:

Input:

abcd@gmail.com

Output:

com
gmail
abcd

For example:

Input	Result
arvijayakumar@rajalakshmi.edu.in	edu.in rajalakshmi arvijayakumar

Answer: (penalty regime: 0 %)

```
s = input().strip()
n = len(s)

# find the first '@' (splits USERNAME from the rest)
at = -1
for i in range(n):
    if s[i] == '@':
        at = i
        break

# find the first '.' after '@' (splits DOMAIN from EXTENSION)
# the extension can hold dots itself, e.g. "edu.in"
dot = -1
for i in range(at + 1, n):
    if s[i] == '.':
        dot = i
        break

username = s[:at]
domain = s[at + 1:dot]
extension = s[dot + 1:]

print(extension)
print(domain)
print(username)
```

	Input	Expected	Got	
✓	abcd@gmail.com	com gmail abcd	com gmail abcd	✓
✓	arvijayakumar@rajalakshmi.edu.in	edu.in rajalakshmi arvijayakumar	edu.in rajalakshmi arvijayakumar	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Assume that the given string has enough memory.

Don't use any extra space(IN-PLACE)

Sample Input 1

a2b4c6

Sample Output 1

aabbbbccccccc

Answer: (penalty regime: 0 %)

```
s = input().strip()
n = len(s)
arr = list(s)    # the "string" buffer we edit in place

# pass 1: find the final length (letter followed by its count)
total = 0
i = 0
while i < n:
    i += 1                              # skip the letter
    num = 0
    while i < n and arr[i].isdigit():   # build the count (handles 12, 105...)
        num = num * 10 + int(arr[i])
        i += 1
    total += num

# grow the buffer so the expanded result fits
if total > n:
    for k in range(total - n):
        arr.append(' ')

# pass 2: read from the back, write from the back
# writing from the end means we never overwrite unread characters
r = n - 1          # read pointer (original string)
w = total - 1      # write pointer (expanded string)

while r >= 0:
    # read the count backwards, e.g. "12" -> 12
    num = 0
    place = 1
    while arr[r].isdigit():
        num = num + int(arr[r]) * place
        place = place * 10
        r -= 1

    letter = arr[r]    # the letter just before the count
    r -= 1

    # write the letter 'num' times, moving left
    for k in range(num):
        arr[w] = letter
        w -= 1

print("".join(arr[:total]))

```

	Input	Expected	Got	
✓	a2b4c6	aabbbbccccccc	aabbbbccccccc	✓
✓	a12b3d4	aaaaaaaaaaaabbbdddd	aaaaaaaaaaaabbbdddd	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Write a Python program to get one string and reverses a string. The input string is given as an array of characters `char[]`.

You may assume all the characters consist of printable ascii characters.

Example 1:

Input:
hello
Output:
olleh

Example 2:

Input:
Hannah
Output:
hannaH

Answer: (penalty regime: 0 %)

```
s = input().strip()
arr = list(s)    # the char[] we reverse in place

# two pointers: swap from both ends moving inward
left = 0
right = len(arr) - 1

while left < right:
    # swap the two characters
    temp = arr[left]
    arr[left] = arr[right]
    arr[right] = temp
    left += 1
    right -= 1

print("".join(arr))

```

	Input	Expected	Got	
✓	hello	olleh	olleh	✓
✓	Hannah	hannaH	hannaH	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Given a string, determine if it is a palindrome, considering only alphanumeric characters and ignoring cases.

Note: For the purpose of this problem, we define empty string as valid palindrome.

Example 1:

```
Input:
A man, a plan, a canal: Panama

Output:
1
```

Example 2:

```
Input:
race a car

Output:
0
```

Constraints:

- `s` consists only of printable ASCII characters.

Answer: (penalty regime: 0 %)

```
s = input()
left = 0
right = len(s) - 1
ok = 1

while left < right:
    # skip non-alphanumeric characters from the left
    while left < right and not s[left].isalnum():
        left += 1
    # skip non-alphanumeric characters from the right
    while left < right and not s[right].isalnum():
        right -= 1

    # compare the two characters, ignoring case
    if s[left].lower() != s[right].lower():
        ok = 0
        break

    left += 1
    right -= 1

print(ok)
```

	Input	Expected	Got	
✓	A man, a plan, a canal: Panama	1	1	✓
✓	race a car	0	0	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.